Clustering and Evaluation in Unsupervised Machine Learning

This section presents the fundamental concepts and principles related to machine learning and clustering analysis. First, different types of learning used in machine learning are introduced, and their roles in data analysis processes are explained. Subsequently, the concept of clustering, its core principles, and the distance measures between data points and clusters are examined in detail.

The section then continues with dimensionality reduction techniques and discusses PCA and t-SNE, which are commonly used methods for visualizing datasets. Methods for evaluating clustering tendency are also introduced, particularly metrics such as the Hopkins statistic, which help reveal hidden structures within the data.

In the section on clustering types and algorithmic approaches, centroid-based, density-based, distribution-based, and hierarchical methods are discussed. This is followed by an explanation of widely used clustering algorithms, including K-Means, DBSCAN, and Gaussian Mixture Models (GMM).

Finally, internal evaluation metrics used to assess clustering performance—such as the Silhouette Coefficient, Davies–Bouldin Index, and Calinski–Harabasz Index—are described in detail.

Overall, this section aims to provide the necessary theoretical foundation for the selection and application of clustering algorithms used in this study.

2.1. Learning Paradigms in Machine Learning

With the rapid increase in **data generation**, expectations regarding data utilization and the quality of this utilization have grown proportionally. In order to meet these increasing demands, **artificial intelligence (AI)** and **machine learning (ML)** techniques are being utilized more than ever before.

In the field of artificial intelligence, there are four main types of learning: **supervised learning**, **unsupervised learning**, **semi-supervised learning**, and **reinforcement learning**. Among these, supervised and unsupervised learning are the most widely used approaches.

One of the fundamental differences between these two approaches lies in whether the data used for training is **labeled** or **unlabeled**. In supervised learning, the training process is carried out using data associated with **target variables (labels)**, whereas in unsupervised learning, the data does not contain any label information. In other words, in supervised learning, the correspondence between inputs and outputs is known in advance, while in unsupervised learning, this relationship is unknown and the model is expected to discover it on its own.

Supervised learning aims to identify **patterns and relationships** between input data and their corresponding **ground truth labels**. Based on these learned relationships, the model can make accurate predictions for new data. Supervised learning methods generally focus on two

main types of problems: **classification** and **regression**. In classification problems, the goal is to assign data to predefined categories, whereas in regression problems, the objective is to predict a **continuous numerical value**.

Unsupervised learning, on the other hand, operates on unlabeled data and aims to uncover **hidden structures, patterns, and relationships** within the dataset. In this approach, the model analyzes similarities and differences among data points to form **natural groups** or meaningful subclusters. The main application areas of unsupervised learning include **clustering**, **association analysis**, and **dimensionality reduction**. Clustering methods group similar data points together, while dimensionality reduction techniques aim to reduce complexity and provide a more manageable representation of the data.

When selecting the appropriate learning type, several key factors must be considered, including whether the dataset is **labeled or unlabeled**, the **nature of the problem**, and the **intended objective** of the solution. In other words, the choice of learning approach should depend on the structure of the data, the problem type, and the desired outcome. Making the correct choice is crucial for improving **model performance**, ensuring the **reliability of results**, and enabling efficient use of **time and computational resources**.

2.2. Clustering Concept and Fundamental Principles

As stated in Section 2.1, clustering is one of the **unsupervised learning** methods. Therefore, it does not require **labeled data (target variables)**. Clustering aims to uncover **hidden patterns, natural structures, and subgroups (segments)** within the data. The primary objective of a clustering algorithm is to group data points with similar characteristics based on their **degree of similarity**. In this process, the similarity or dissimilarity between samples is typically computed using various **distance measures**.

As a result of the clustering process, the dataset is divided into **K distinct groups (clusters)**. The goal is to achieve **high intra-cluster similarity (low variance within clusters)** and **low inter-cluster similarity (high separation between clusters)**. A visual representation of this concept is shown in **Figure 2.1**.

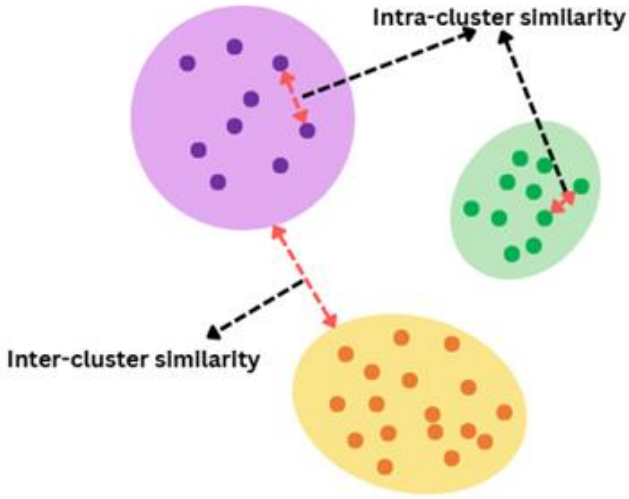

Figure 2.1. Visualization of intra-cluster and inter-cluster relationships

Clustering methods are widely used in various application domains, including **marketing and customer segmentation**, **image processing**, **anomaly detection**, **recommendation systems**, and **document classification**. In addition, they have significant applications in fields such as **healthcare**, **finance**, and **geographic information systems (GIS)**.

2.2.1 Distance Measures

Distance measures can generally be categorized into two main groups: **distance measures between data points** and **distance measures between clusters**.

Distance measures between data points quantify the distance between two points in the **feature space**. These measurements allow the determination of the degree of **similarity or dissimilarity** between data points, enabling the assignment of data into appropriate clusters. Since different distance metrics vary in their computational approaches, the choice of metric can directly influence the **clustering results** and the relationships between clusters. The impact of these differences on clustering performance has been clearly demonstrated in the study by **Kumar et al. (2005)**, titled *Performance Evaluation of Distance Metrics in the Clustering Algorithms*. Their findings show that different distance measures can lead to significant variations across clustering techniques and data structures.

On the other hand, **distance measures between clusters** play a crucial role, particularly in **hierarchical clustering algorithms**. In such methods, it is necessary to define the distance between clusters. The way clusters are merged or separated is determined by **linkage methods**, which are based on distances between cluster members.

2.2.1.1 Common Distance Measures Between Data Points

Euclidean Distance is the most widely used metric for continuous numerical data and represents the straight-line distance (L2 norm) between two points:

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Manhattan Distance (City Block Distance) measures the distance between two points as the sum of the absolute differences of their coordinates:

$$d(x, y) = \sum_{i=1}^{n} | x_i - y_i |$$

Minkowski Distance is a generalized distance metric that includes both Euclidean and Manhattan distances as special cases:

$$d(x,y) = (\sum_{i=1}^{n} | x_i - y_i |^p)^{1/p}$$

Cosine Distance measures the angle between two vectors rather than their absolute distance, focusing on **directional similarity** rather than magnitude:

$$\text{Cosine Similarity} = \frac{X \cdot Y}{\| X \| \| Y \|}$$
$$d_{\cos}(X,Y) = 1 - \text{Cosine Similarity}$$

Correlation-based Distance measures how two variables change together:

$$d(x,y) = 1 - \text{corr}(x,y)$$

2.2.1.2 Distance Measures Between Clusters

Single Linkage: The distance between two clusters is defined as the **minimum distance** between any two data points belonging to different clusters.

$$D(A,B) = \min_{x \in A,\, y \in B} d(x,y)$$

Complete Linkage: The distance between two clusters is defined as the **maximum distance** between data points in the clusters.

$$D(A,B) = \max_{x \in A,\, y \in B} d(x,y)$$

Average Linkage: The distance between two clusters is calculated as the **average distance** between all pairs of data points from the two clusters.

$$D(A,B) = \frac{1}{| A || B |} \sum_{x \in A} \sum_{y \in B} d(x,y)$$

Ward Linkage: This method aims to merge clusters in a way that results in the **minimum increase in intra-cluster variance** (i.e., minimizes the increase in the sum of squared errors). In other words, it preserves **cluster homogeneity** at each step.

$$\Delta E = \frac{| A || B |}{| A | + | B |} \| \mu_A - \mu_B \|^2$$

2.2.2 Dimensionality Reduction Techniques

Dimensionality reduction is a fundamental technique in the **machine learning (ML)** process that simplifies datasets by reducing the number of input variables or features. In today's **big data era**, where data is generated at an unprecedented rate, managing large volumes of information efficiently has become increasingly challenging. Therefore, dimensionality reduction strategies have become essential for improving **computational efficiency** and enhancing the model's **generalization capability**

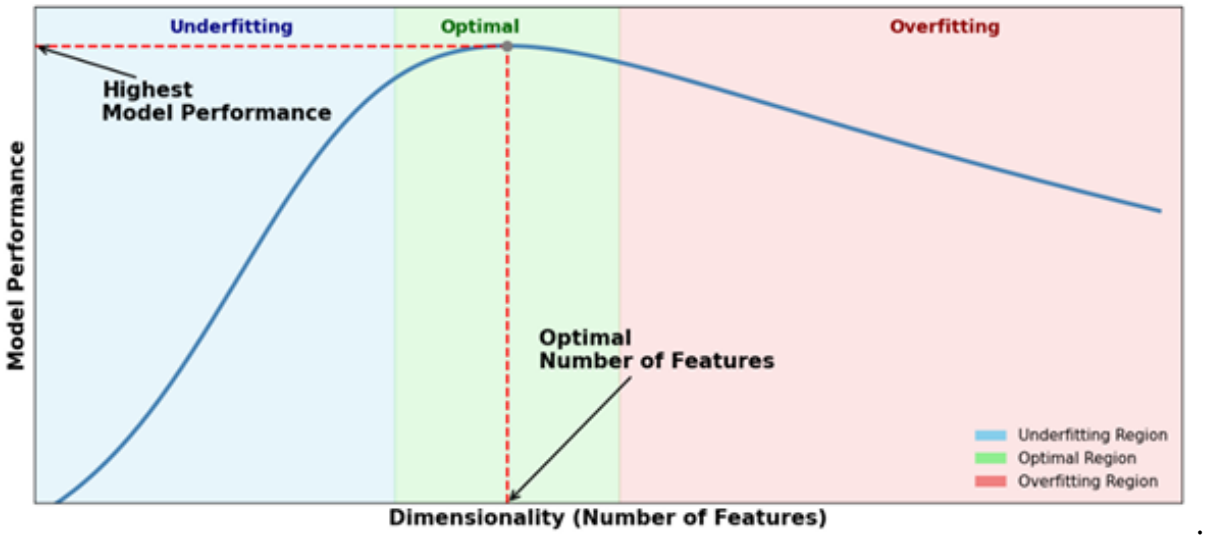

Figure 2.2. Effect of feature dimensionality on model performance

High-dimensional datasets often contain hundreds or even thousands of features. In such cases, a phenomenon known as the **curse of dimensionality** arises. This refers to the degradation in algorithm performance and efficiency as the number of dimensions increases. As dimensionality grows, data becomes more **sparse**, computational costs increase, and the learning process slows down.

Dimensionality reduction techniques transform data into a lower-dimensional space while preserving its essential information. This leads to reduced computational cost, faster model training, improved visualization, and a lower risk of **overfitting**. However, since some features are removed, a slight decrease in model performance may occur, which is considered an acceptable trade-off. Therefore, selecting or extracting the most relevant features is critically important .

Dimensionality reduction can be performed using two main approaches:

- **Feature Selection:** Identifies the most relevant features from the dataset and removes unnecessary or less informative ones. In this approach, original features are preserved, and only a subset is selected. Common methods include **filter**, **wrapper**, and **embedded** techniques.
- **Feature Extraction:** Generates new features by combining or transforming the original ones, thereby projecting the data into a lower-dimensional space. Unlike

feature selection, this approach reconstructs the feature space. **PCA** and **t-SNE** are widely used examples of this method.

In this study, only **Principal Component Analysis (PCA)** and **t-distributed Stochastic Neighbor Embedding (t-SNE)** are employed as dimensionality reduction techniques.

2.2.2.1 Principal Component Analysis (PCA)

PCA is one of the most commonly used techniques for reducing the dimensionality of a dataset. It performs **linear dimensionality reduction**, meaning that it projects data into a lower-dimensional space by utilizing linear relationships among the original features. Since PCA follows a **feature extraction** approach, it generates **principal components**, which are combinations and transformations of the original features. These components aim to preserve the underlying **information and patterns** within the dataset.

One of the main disadvantages of PCA is that the resulting components can be difficult to interpret, as they are combinations of the original variables. Nevertheless, PCA improves computational efficiency and enhances model performance by reducing noise while lowering dimensionality.

The PCA process consists of the following steps:

- **Standardization:** Data is normalized to ensure that each variable contributes equally. This step addresses PCA's sensitivity to feature scales.
- **Covariance Matrix Computation:** Measures how variables vary from the mean and how they relate to each other.
- **Eigenvalue and Eigenvector Calculation:** Identifies new axes (**eigenvectors**) that maximize variance while remaining orthogonal.
- **Ranking and Selection:** Eigenvectors are ranked based on their corresponding **eigenvalues**, which represent their variance contribution.
- **Feature Vector Construction:** Selected eigenvectors are combined to form the principal components.
- **Transformation:** The original data is projected onto the new lower-dimensional space for further analysis or machine learning tasks.

2.2.2.2 t-Distributed Stochastic Neighbor Embedding (t-SNE)

t-SNE is a powerful **non-linear dimensionality reduction** technique used to visualize high-dimensional and complex data in two or three dimensions. Unlike linear methods, non-linear dimensionality reduction captures **complex, non-linear relationships** among data points.

t-SNE focuses on preserving **local similarities** between data points in the lower-dimensional space. In contrast, PCA aims to maximize global variance and preserve large pairwise distances. By maintaining local structure, t-SNE ensures that points belonging to the same

cluster or having similar relationships remain close to each other in the transformed space. Therefore, it is particularly effective for visualizing datasets with **complex and non-linear structures**.

The working mechanism of t-SNE involves the following steps:

- **High-dimensional similarity computation:** Calculates pairwise similarity probabilities, where nearby points have higher similarity.
- **Projection to lower-dimensional space:** Initializes points randomly and rearranges them to reflect high-dimensional relationships.
- **Loss function minimization:** Minimizes a divergence measure (e.g., **Kullback–Leibler divergence**) between high- and low-dimensional similarities.
- **Iterative optimization:** Updates point positions iteratively until convergence.

One of the main drawbacks of t-SNE is its **high computational cost** compared to PCA. However, its ability to provide intuitive and meaningful visualizations makes it highly valuable for understanding complex datasets.

2.2.3 Evaluating Clustering Tendency

Before applying any clustering algorithm to a dataset, it is essential to verify whether the data actually contains a **meaningful cluster structure**. If a dataset is not naturally prone to clustering, applying clustering algorithms may result in **artificially formed clusters** that do not carry any statistical or semantic significance. Therefore, evaluating the **clustering tendency** of a dataset is a critical preliminary step before conducting clustering analysis. This evaluation helps determine whether an inherent grouping structure truly exists within the data.

There are two main approaches for assessing clustering tendency:

- **Statistical methods:** The most well-known method in this category is the **Hopkins statistic**, which tests whether data points are randomly distributed and provides a quantitative measure of clustering tendency.
- **Visual methods:** This approach utilizes the **Visual Assessment of Cluster Tendency (VAT)** algorithm. It generates a heatmap based on the distance matrix, allowing the clustering structure of the data to be visually examined.

2.2.3.1 Hopkins Statistic

The **Hopkins statistic** is one of the most fundamental and practical statistical methods used to determine whether a dataset has a **clusterable structure**. In other words, it evaluates whether the data is randomly distributed or exhibits a tendency to form clusters.

This method compares the **nearest neighbor distances** of actual data points with those of randomly generated points within the same space and scale. Through this comparison, it

determines whether the observed similarity structure arises by chance or reflects a genuine clustering pattern.

The Hopkins statistic is computed through the following steps:

- Randomly select **n observations** from the dataset.
- Generate **n random points** within the same feature space.
- For each random point, compute the distance to its nearest real data point ($\mathbf{y_i}$).
- For each selected real data point, compute the distance to its nearest neighbor ($\mathbf{x_i}$).
- The Hopkins statistic is then calculated as:

$$H = \frac{\sum_{i=1}^{n} y_i}{\sum_{i=1}^{n} x_i + \sum_{i=1}^{n} y_i}$$

The value of **H** ranges between 0 and 1 and can be interpreted as follows:

- **H ≈ 0.5** → Data points are randomly distributed; no clustering tendency.
- **H > 0.5** → The dataset exhibits a clustering tendency.
- **H < 0.5** → Data points are uniformly or regularly distributed; no meaningful clustering tendency.
- **H > 0.75** → Strong clustering tendency (approximately 90% confidence level).

It is important to note that the Hopkins statistic does not determine the **number of clusters**; it only assesses the **existence of a clustering structure**. Therefore, it is commonly used as a **preliminary test** before applying clustering algorithms.

The fundamental hypotheses of the Hopkins test are:

- **H_0 (Null Hypothesis):** The dataset is uniformly distributed; no meaningful cluster structure exists.
- **H_1 (Alternative Hypothesis):** The dataset is not uniformly distributed; it contains meaningful clusters.

As the Hopkins value approaches **1**, the strength of the clustering structure increases, whereas values around **0.5** indicate a random and non-clustered distribution. For this reason, the Hopkins statistic is a highly useful indicator for evaluating whether clustering analysis is appropriate for a given dataset.

2.2.4 Types of Clustering

Although there are many clustering algorithms, they can be broadly categorized into two main types based on how data points are assigned to clusters: **Hard Clustering** and **Soft (Fuzzy)**

Clustering. This distinction helps us understand the relationship between clusters and their members.

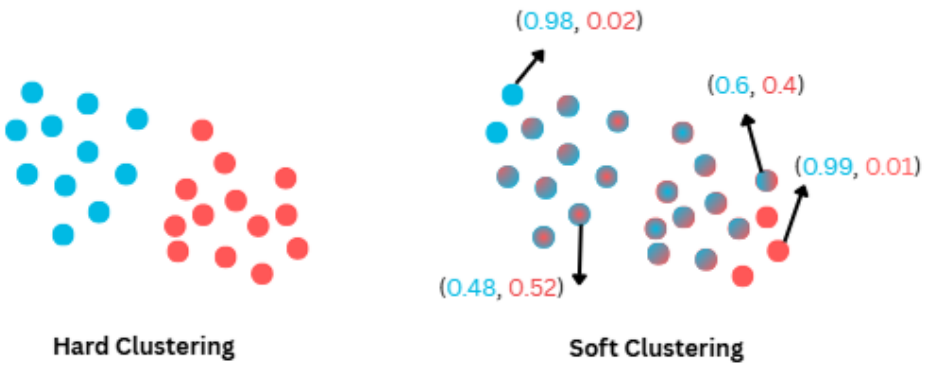

Figure 2.3. Comparison of Hard and Soft Clustering

In **Hard Clustering**, each data point is assigned to **only one cluster**. In this approach, cluster boundaries are **clear and well-defined**, and there is no overlap between clusters. The simplicity of implementation and the interpretability of results have contributed to the widespread use of algorithms based on hard clustering. This approach performs well when the data distribution is clearly separable. However, it faces challenges in datasets where cluster boundaries are **ambiguous or overlapping**, as it ignores the possibility that a data point may belong to multiple clusters.

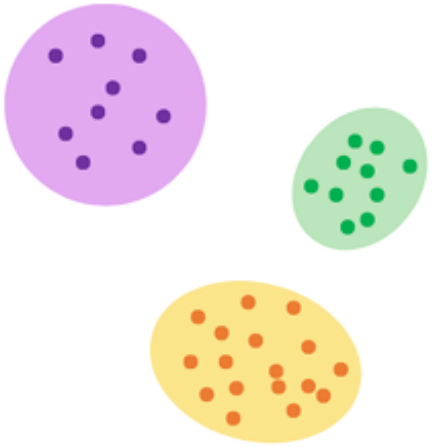

Figure 2.4. Example of Hard Clustering

In contrast, **Soft (or Fuzzy) Clustering** allows data points to belong to **multiple clusters**, with each point having a certain **degree of membership** for each cluster. Instead of assigning data points in a binary manner (belonging or not belonging), this approach represents each point with a **probability or membership value** relative to each cluster.

Unlike hard clustering, soft clustering permits **overlapping clusters**, making it more suitable for datasets with **uncertain or non-distinct boundaries**. By assigning probabilistic memberships, it can better capture **uncertainty** in the data. However, this flexibility comes

with certain drawbacks, including **higher computational cost** and increased complexity in interpreting the results due to partial memberships.

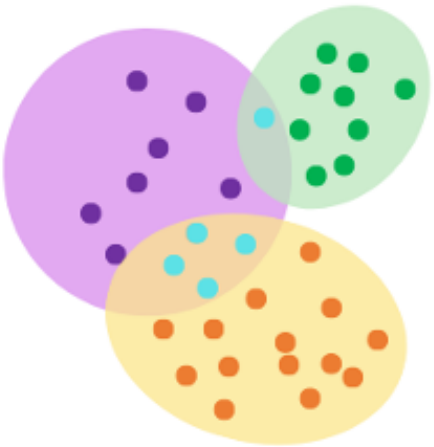

Figure 2.5. Example of Soft Clustering

In summary, hard clustering provides a **simpler and more interpretable** framework by assigning each data point to a single cluster, whereas soft clustering offers a **more flexible and realistic** approach by incorporating probabilistic assignments. The choice between these approaches depends on factors such as the **data distribution**, the **underlying structure of the dataset**, and the **nature of the problem** being addressed.

2.2.5 Clustering Algorithm Methods

As emphasized in the previous sections, it is possible to make informed assumptions about which clustering method or algorithm will be more suitable and effective based on the **structural characteristics of the data**. This section discusses the fundamental clustering approaches and the principles according to which these methods form clusters. A widely accepted approach in the literature is to classify clustering algorithms into five main categories: **centroid-based**, **hierarchical**, **density-based**, **grid-based**, and **model-based** methods. This classification allows for a systematic evaluation of the **working mechanisms** and **strengths** of different algorithms.

The taxonomy used in this study is based on this general framework. **Figure 2.6** presents an overview of this taxonomy and illustrates example algorithms corresponding to each clustering type.

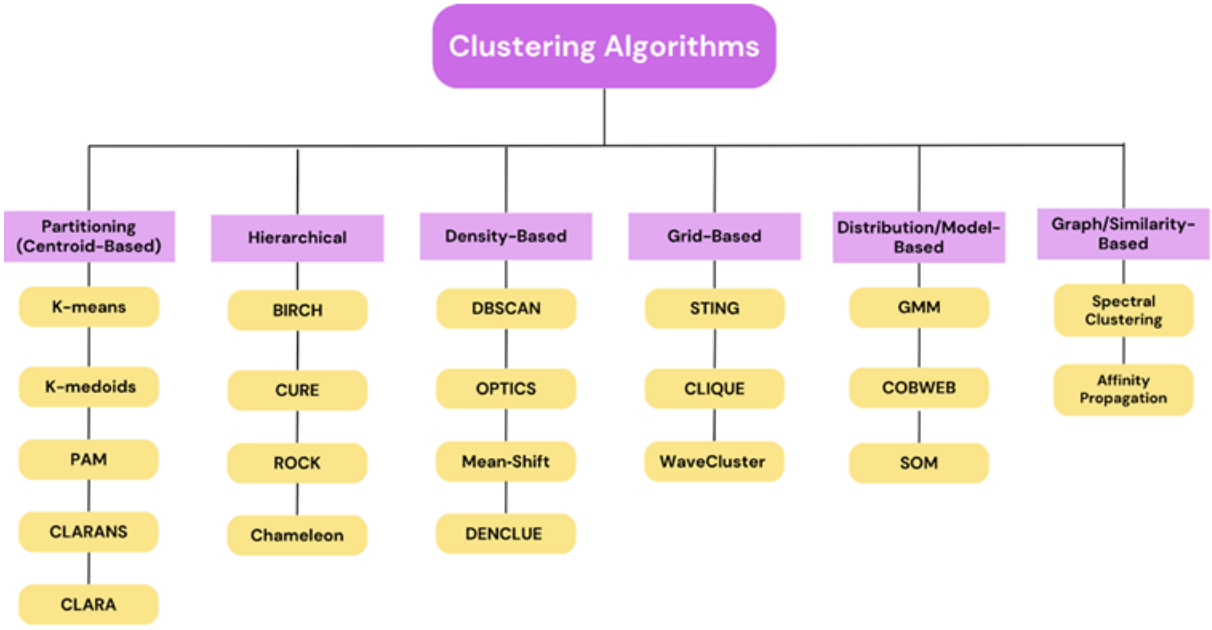

Figure 2.6. Taxonomy of clustering algorithms

2.2.5.1 Centroid-Based Clustering

This is one of the most well-known and widely used clustering approaches. In this method, data points are grouped based on their proximity to multiple **centroid points**. Each data point is assigned to a cluster according to the **squared distance** to its corresponding centroid. These methods are generally **fast and efficient**, but they are sensitive to **initial parameter selection**.

The dataset is partitioned into **K clusters**, where K is a predefined parameter, and each data point is assigned to only one cluster (**hard clustering**). Determining the optimal value of **K** is crucial for the success of the method. Common techniques used for this purpose include the **Elbow Method** and the **Silhouette Coefficient**.

The Elbow Method identifies the optimal K by analyzing the point where the decrease in the **sum of squared errors (SSE)** slows down significantly. The Silhouette Coefficient evaluates clustering quality by comparing the similarity of each point within its cluster to its distance from other clusters. The combination of these methods helps improve both the selection of the optimal number of clusters and the performance of centroid-based algorithms such as **K-Means**, which is considered the most representative algorithm of this approach.

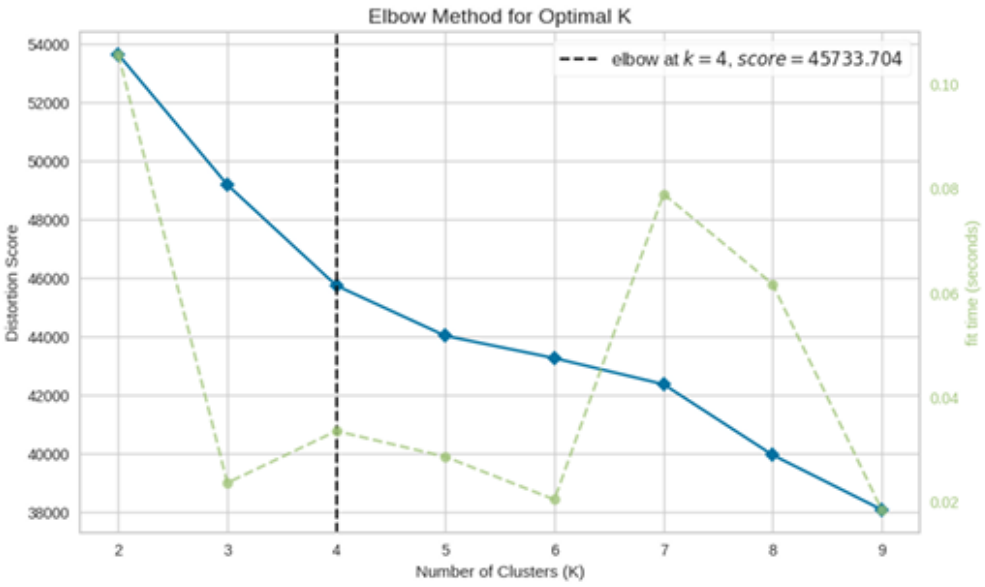

Figure 2.7. Elbow method

2.2.5.2 Density-Based Clustering

In density-based approaches, clustering is performed by grouping together data points that are **closely packed in high-density regions**, separated by areas of **low density** (noise or outliers). In simple terms, these algorithms identify dense regions in the data space and define them as clusters.

Unlike centroid-based methods, clusters in density-based approaches are not restricted to spherical or circular shapes; they can take on **arbitrary and complex forms**. This makes them particularly effective for datasets with **irregular geometries** or **noise**. Another important advantage is their ability to **detect and exclude outliers**, preventing them from being assigned to clusters.

Additionally, these methods do not require a predefined number of clusters (K), which increases model flexibility. However, selecting appropriate parameters is critical; incorrect parameter choices may lead to either excessive fragmentation into small clusters or merging of clusters, resulting in loss of meaningful structure. Well-known examples of density-based algorithms include **DBSCAN** and **OPTICS**, which often produce more realistic and meaningful results compared to centroid-based methods, especially for complex datasets.

2.2.5.3 Distribution-Based Clustering

This approach assumes that the data points are generated from underlying **probability distributions** (e.g., Gaussian, Binomial). Accordingly, each data point is assigned to clusters based on the **probability of belonging** to each cluster. Therefore, these methods adopt a **soft clustering** approach and allow **overlapping clusters**.

The most well-known example is the **Gaussian Mixture Model (GMM)**. GMM assumes that each cluster corresponds to a Gaussian distribution and that the data is generated from a

mixture of these distributions. This allows the model to better capture **uncertainty** and represent clusters with different distributional characteristics. However, selecting the correct number of components (clusters) and accurately estimating model parameters are critical for achieving good performance .

2.2.5.4 Hierarchical Clustering

Hierarchical clustering assumes that data points that are closer to each other have a **stronger relationship**, while distant points have a weaker one. In this method, clustering is performed based on **distance and neighborhood relationships** among data points.

The result of this process is a tree-like structure called a **dendrogram**, which visually represents how clusters are merged or split. Due to this representation, the method is referred to as hierarchical clustering.

The distance between clusters is calculated using **linkage methods**, such as **single linkage** and **complete linkage**, as described in Section 2.2.1.2. Hierarchical clustering can be implemented in two main ways:

- **Agglomerative (Bottom-Up):** Each data point starts as its own cluster, and clusters are iteratively merged.
- **Divisive (Top-Down):** All data points start in a single cluster, which is then recursively split into smaller clusters.

Figure 2.8 illustrates how different hierarchical linkage methods behave across various synthetic datasets. Each linkage method groups data differently depending on the **shape, density, and structure** of the clusters.

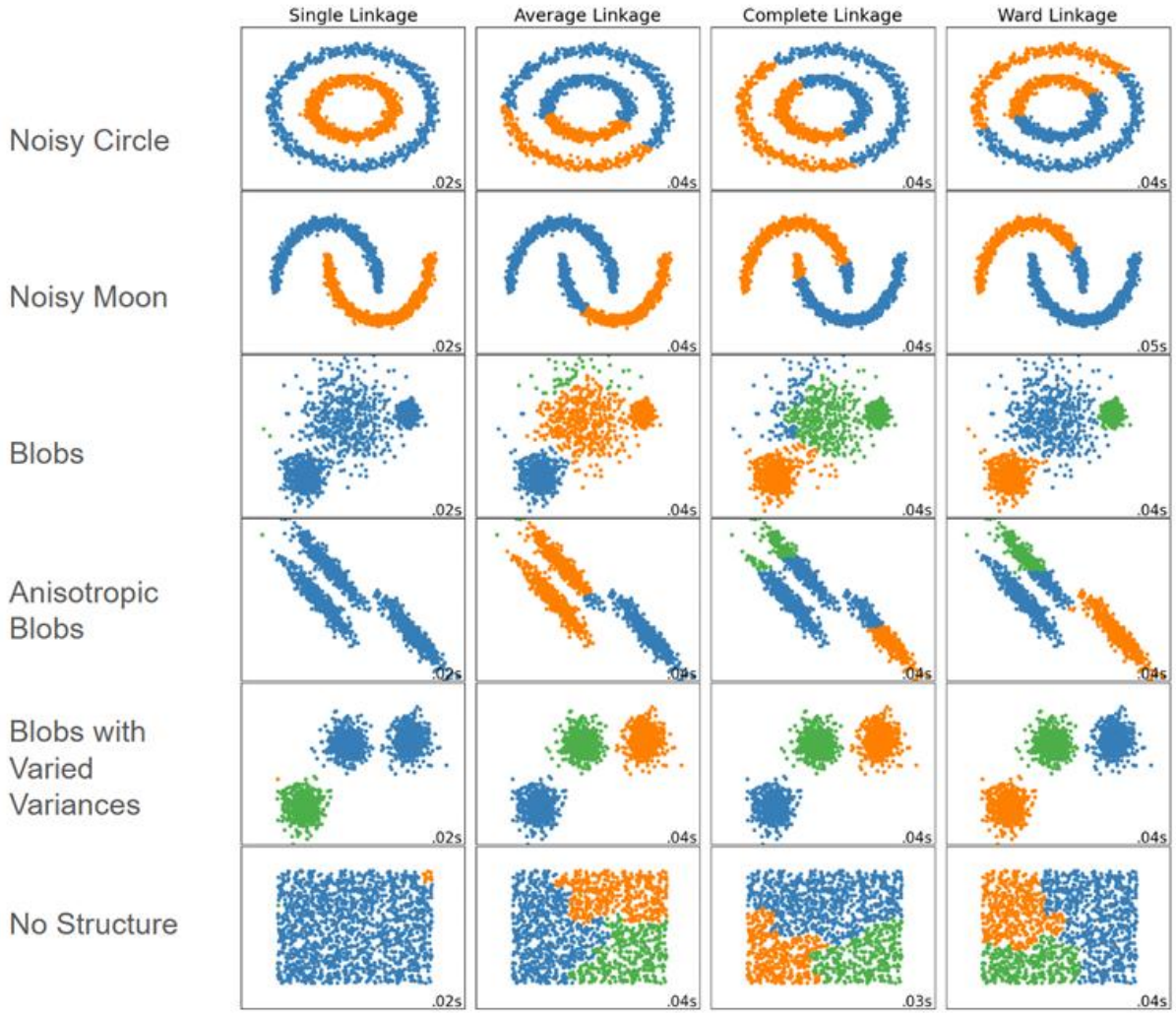

Figure 2.8. Comparison of hierarchical linkage methods on synthetic datasets

The **Ward linkage** method aims to minimize **intra-cluster variance**, producing more homogeneous clusters. In **Figure 2.9**, the hierarchical clustering process using Ward linkage is visualized through both data points and the corresponding dendrogram. The left panel shows clusters with convex boundaries and marked cluster centers, while the right panel presents the dendrogram illustrating the hierarchical merging process.

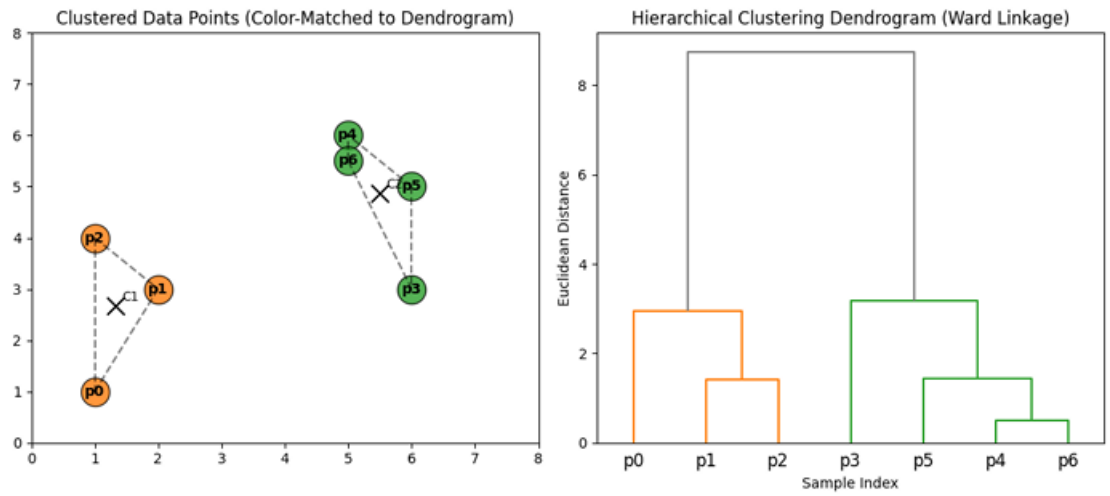

Figure 2.9. Visualization of hierarchical clustering using Ward linkage

2.2.6 Clustering Algorithms

Over time, various clustering algorithms have been developed, each based on different **assumptions**, **strengths**, and **computational strategies**. Some of these algorithms focus on grouping data points around specific **centroids**, while others form clusters based on **density**, **hierarchical structures**, or **probabilistic distributions**. In order to analyze the performance differences and suitability of these approaches for different data structures, a representative algorithm from each category is selected and evaluated using **internal validation metrics**.

In this study, the following models are used:

- **K-Means** (centroid-based)
- **DBSCAN** (density-based)
- **Agglomerative Clustering** (hierarchical)
- **Gaussian Mixture Model (GMM)** (distribution-based)

Figure 2.10 illustrates how different clustering algorithms perform on various synthetic datasets with distinct geometric and statistical properties. It highlights that some algorithms are better at handling **non-linear or irregular cluster shapes** than others.

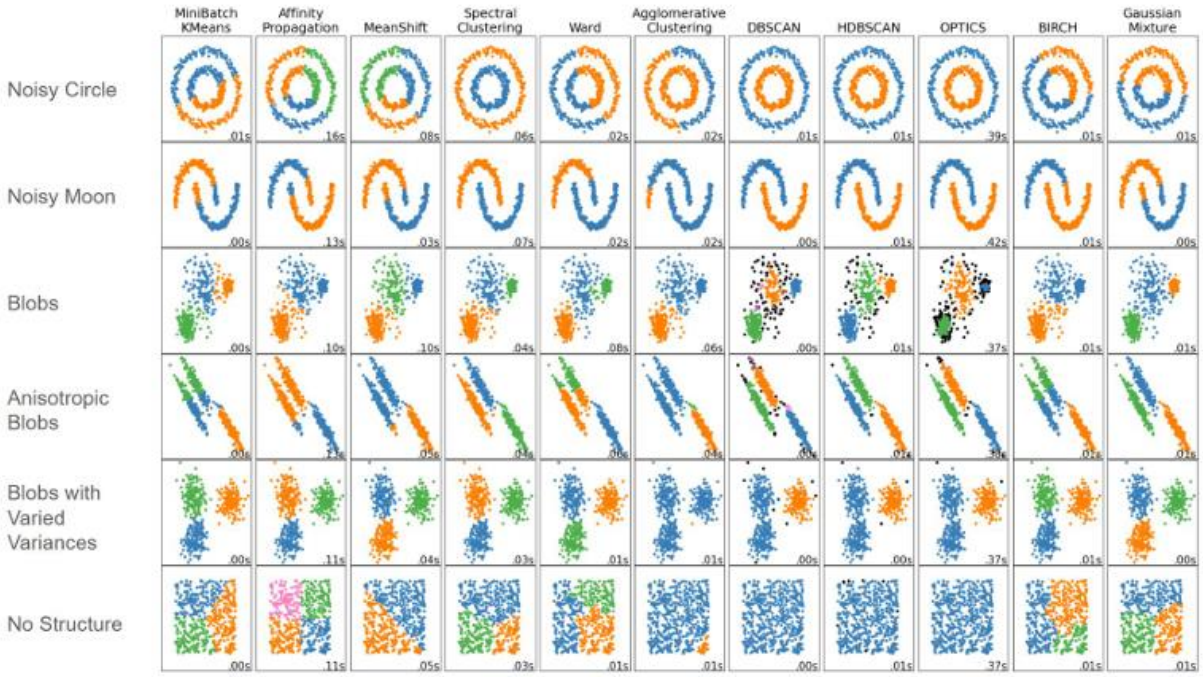

Figure 2.10. Comparison of clustering algorithms on synthetic datasets

2.2.6.1 K-Means

K-Means is one of the most widely used and popular algorithms for clustering analysis. As discussed earlier, it is an example of a **hard clustering** method, meaning that each data point belongs to **only one cluster**, with no overlap between clusters.

As a **centroid-based** method, K-Means assigns data points to the nearest **cluster centroids**. The primary objective of the algorithm is to minimize the **distance between data points and their respective centroids**, thereby reducing **intra-cluster variance** as much as possible. For this purpose, distance metrics such as **Euclidean** or **Manhattan distance** are used.

The algorithm begins with a user-defined parameter **K**, which specifies the number of clusters. Initially, centroids are selected randomly, and then iteratively updated. At each step, new centroids are computed as the **mean of the data points** assigned to each cluster. This process continues until the centroid values stabilize.

Selecting the optimal **K value** is critical for the success of the algorithm. Common methods used for this purpose include the **Elbow Method** and the **Silhouette Score**. However, K-Means is highly sensitive to **outliers**, as extreme values can significantly distort cluster centers.

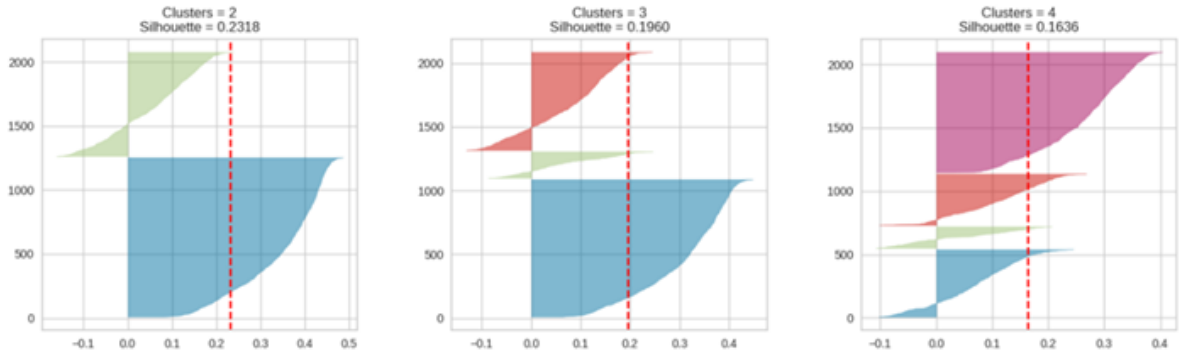

Figure 2.11. Silhouette Score

Several variants of K-Means exist, including **Mini-Batch K-Means**, **K-Medians / K-Medoids**, and **K-Means++**. Although K-Means is fast, simple, and easy to implement, its dependence on the initial parameter **K** and sensitivity to outliers are its main limitations.

2.2.6.2 DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a **density-based clustering algorithm**. As the name suggests, it groups together data points that are closely packed in **high-density regions**, while labeling points in sparse regions as **noise**.

This characteristic makes DBSCAN particularly effective for **anomaly and outlier detection**. The algorithm separates clusters based on low-density regions, allowing it to clearly distinguish between dense areas. It performs especially well when clusters have **irregular shapes** or when the number of clusters is not known in advance.

After clustering, DBSCAN classifies data points into three categories:

- **Core points:** Points located in dense regions with a sufficient number of neighbors
- **Border points:** Points that are not dense enough to form a core but lie within the neighborhood of a core point
- **Noise points:** Isolated points that do not belong to any cluster

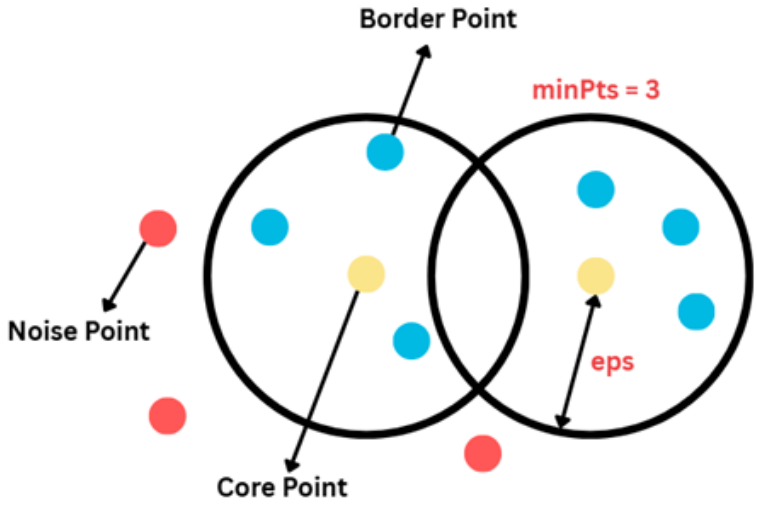

Figure 2.12. DBSCAN point types

DBSCAN relies on two key parameters:

- **ε (eps):** Defines the radius of the neighborhood around a data point
- **minPts (min_samples):** Specifies the minimum number of points required to form a dense region

If a data point has at least **minPts neighbors within its ε-radius**, it is classified as a **core point**; otherwise, it is considered an **outlier**. This process is repeated for all data points to form clusters .

2.2.6.3 Gaussian Mixture Model (GMM)

The **Gaussian Mixture Model (GMM)** is a **probabilistic (distribution-based)** clustering method. It assumes that the data is generated from a mixture of multiple **Gaussian (normal) distributions**, each representing a cluster. As a data point moves further away from a distribution, its probability of belonging to that distribution decreases.

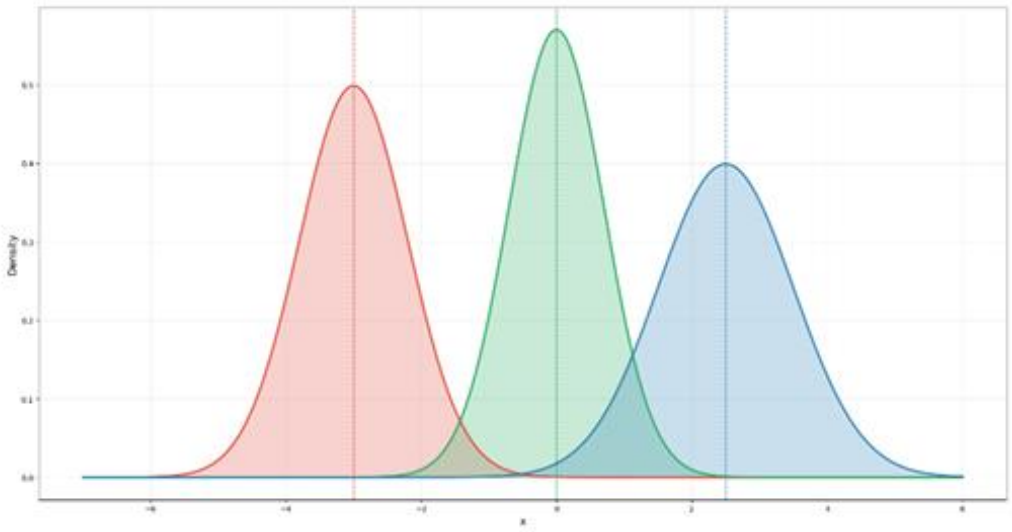

Figure 2.13. Representation of different normal distributions

Unlike K-Means, GMM performs **soft clustering**, meaning that each data point is assigned a **probability of belonging** to each cluster rather than being assigned to a single cluster. This allows GMM to produce more **flexible and realistic results**, especially for datasets with **overlapping or complex structures**. Additionally, unlike K-Means, clusters are not restricted to spherical shapes.

Each Gaussian component is defined by three key parameters:

- **Mean (μ):** Represents the center of the cluster
- **Covariance (Σ):** Defines the spread and shape of the cluster
- **Mixing coefficient (π):** Indicates the proportion or importance of the cluster within the dataset

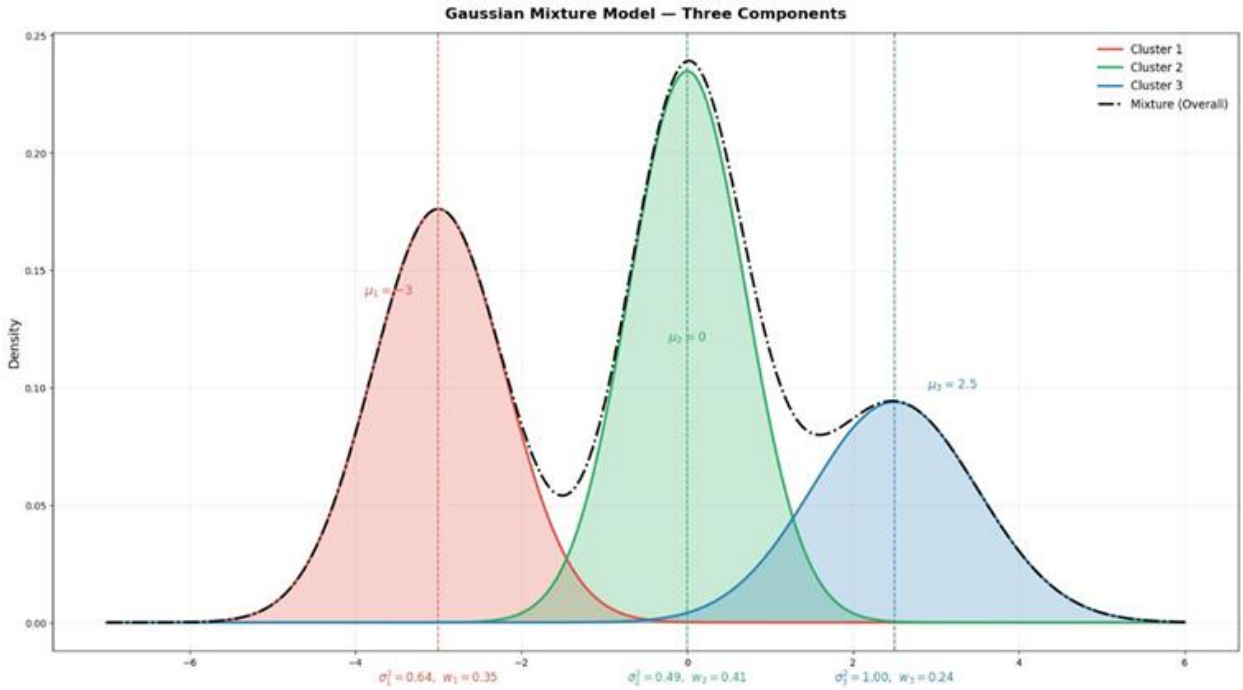

Figure 2.14. Visualization of Gaussian components in GMM

GMM uses the **Expectation-Maximization (EM)** algorithm to optimize these parameters. The EM algorithm iteratively improves probability estimates to maximize the **likelihood** of the model. The process consists of the following steps:

- **Initialization:** Assign initial values for μ, Σ, and π
- **Expectation Step (E-step):** Compute the probability that each data point belongs to each cluster
- **Maximization Step (M-step):** Update the Gaussian parameters based on these probabilities

- **Convergence:** Repeat the E and M steps until the likelihood converges

As a result, GMM provides a flexible clustering structure by estimating both the **cluster distributions** and the **membership probabilities** of each data point.

2.2.7 Clustering Evaluation Metrics

Evaluating whether the resulting clusters are **meaningful and valid** is a critical aspect of clustering analysis. In **unsupervised learning**, unlike supervised learning, there is no **ground truth (target labels)** available. Therefore, clustering performance cannot be directly measured using standard metrics such as **accuracy** or **recall**.

However, clustering algorithms can still be used in labeled datasets for tasks such as **pre-labeling** or grouping unlabeled samples. In pure clustering analysis, though, labeled data is typically not used.

There are two main types of evaluation metrics for assessing clustering quality:

- **Internal metrics:** These evaluate clustering quality based solely on **data features and cluster assignments**. They are the most commonly used metrics in real-world applications where labels are unavailable. Examples include the **Silhouette Score**, **Davies–Bouldin Index**, and **Calinski–Harabasz Index**.
- **External metrics:** These measure how well the clustering results match known **ground truth labels**. Since labeled data is rarely available in practice, these metrics are mostly used for **academic comparisons**. Examples include **Adjusted Rand Index (ARI)**, **Normalized Mutual Information (NMI)**, and **Fowlkes–Mallows Index**.

In this study, **internal evaluation metrics** are used to assess clustering performance.

2.2.7.1 Internal Metrics

Internal metrics evaluate the **structural quality** and **separation** of clusters based solely on the positions of data points and their cluster assignments. These metrics do not require any external reference or labeled data. Instead, they quantify both **intra-cluster similarity** and **inter-cluster separation**.

In a well-formed clustering structure:

- Data points within the same cluster should be **close to each other**
- Data points from different clusters should be **far apart**

Thus, internal metrics serve as essential tools for **objective evaluation** and comparison of clustering results.

2.2.7.1.1 Silhouette Coefficient

The **Silhouette Coefficient** is an internal evaluation metric that measures how well each data point fits within its assigned cluster and how clearly clusters are separated.

The main idea is that a data point should be **close to points in its own cluster** (low intra-cluster distance) and **far from points in other clusters** (high inter-cluster distance).

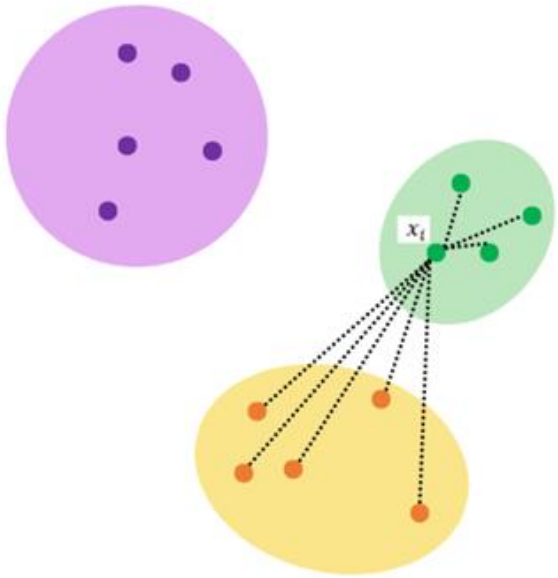

Figure 2.15. Calculation process of the Silhouette Index

The calculation steps are as follows:

- For each data point:
    - Compute the average distance to other points in the same cluster (**A**)
    - Compute the average distance to the nearest neighboring cluster (**B**)
- The Silhouette value is calculated as:

$$s = \frac{B - A}{\max(A, B)}$$

Where:

- **s** → Silhouette value
- **A** → intra-cluster distance
- **B** → nearest-cluster distance

The overall Silhouette Score is obtained by averaging the values across all data points. The score ranges between **-1 and +1**:

- **Close to +1** → well-separated and correctly clustered data
- **Close to 0** → data points near cluster boundaries

- **Close to -1** → likely misclassified points

A higher Silhouette Score indicates **better clustering quality** and is also commonly used to determine the **optimal number of clusters (K)**.

2.2.7.1.2 Davies–Bouldin Index

The **Davies–Bouldin Index (DBI)** is an internal metric that measures the **similarity between clusters** and their degree of separation. It evaluates how distinct each cluster is relative to others.

The index is computed by comparing **intra-cluster dispersion** with **inter-cluster distances**:

- Compute average intra-cluster distance (**S**)
- Compute distance between cluster centers (**M**)

The similarity ratio between clusters *i* and *j* is:

$$R_{ij} = \frac{S_i + S_j}{M_{ij}}$$

The DBI is calculated as:

$$DBI = \frac{1}{k}\sum_{i=1}^{k} \max_{j \neq i} (R_{ij})$$

A **lower DBI value** indicates better clustering, meaning clusters are more **compact and well-separated**.

2.2.7.1.3 Calinski–Harabasz Index

The **Calinski–Harabasz Index (CHI)** is based on the ratio of **between-cluster dispersion** to **within-cluster dispersion**. It assigns higher values when clusters are well separated and internally compact.

It is defined as:

$$CHI = \frac{Tr(B_k)/(k-1)}{Tr(W_k)/(n-k)}$$

Where:

- **$Tr(B_k)$** → between-cluster variance
- **$Tr(W_k)$** → within-cluster variance

- **n** → number of data points
- **k** → number of clusters

A **higher CHI value** indicates better clustering performance, with strong separation and low internal variance.

Overall Evaluation

Based on internal metric results, it becomes easier to determine whether a dataset contains a **meaningful clustering structure**.

- **Silhouette Score** evaluates both **compactness and separation**
- **Calinski–Harabasz Index** measures overall **cluster dispersion structure**
- **Davies–Bouldin Index** assesses **inter-cluster similarity**

A good clustering result is typically characterized by:

- **High Silhouette Score**
- **High Calinski–Harabasz value**
- **Low Davies–Bouldin value**

When these conditions are met, the clustering model can be considered **well-balanced, well-separated, and structurally consistent**.